

FShiNG

GAME DESIGN DOCUMENT



Team Liam - Lead Programmer / Game Designer / Gameplay Tester; Saxon - Programmer / Visual Designer / Gameplay Tester; Harrison - Progression Writer / Documentation / Gameplay Tester; David - Sound Designer / Marine Specialist.

Prepared for the Year 10 Computing Game Design Incursion. Version 0.9.0 (PR #114), 29 August 2026.

Elevator pitch

A 45-60 second classroom pitch

One-line pitch Read the market, catch real fish, and choose the harbour that pays more.

FSHING is a side-on fishing market game where every trip begins with a decision. You check the day's price, track a real species, sail to its habitat, and manage the line while the fish fights back. The catch only becomes profit when you choose where to sell it.

The market and fishing systems depend on each other. Better line depth reaches more valuable fish. More cargo makes a trip worth taking. A stronger engine shortens the crossing, but prices still move each day. The result is a short browser game about observation, timing, and trade-offs, not a fishing quiz.



Real gameplay: the player reads a seven-day price history before tracking Bluegill.

Say it like this

“In FSHING, the lake is both a fishing ground and a marketplace. You read today's quote, track a species, sail to the right habitat, and work the reel without snapping the line. Then you decide where to sell and what to upgrade next. It uses real fish, moving prices, and a short tutorial so a new player can understand the loop by playing it.”

Main character

The working boat is the player's visible character



Who the player controls

FSHING does not use a named human avatar. The player's character is a small, weathered working boat seen in side profile. Its dark teal hull and cream wheelhouse match the lake interface. The cargo crate, hook crane, smoke stack, and warm roof lantern make its purpose readable without dialogue.

Character through mechanics

- Movement: the boat accelerates, brakes, turns, docks, and can later use a heat-limited boost.
- Work: it carries the hook, line, reel, and catches. The crane faces the fishing side of the vessel.
- Growth: cargo, engine, line depth, reel power, boost, and Beach access change what the boat can do.
- Identity: practical rather than heroic - a small vessel earning its way across a large lake.

Supporting illustration generated from the production boat sprite and lake art direction. All gameplay evidence elsewhere in this document uses screenshots from the live build.

Game concept and learning purpose

Concept

The player runs a fishing boat between two harbours. Each harbour posts its own seeded daily quote for every discovered species. The player tracks a fish, sails to its habitat, catches it through a reel-and-release fight, and sells the whole catch at the displayed quote.

Learning purpose

- Read a seven-day graph and compare today's value with recent prices.
- Recognise that the nearest harbour is not always the best sale.
- Connect fish species to habitats and depth bands.
- Choose upgrades by explaining a trade-off: capacity, travel speed, reach, or reel control.
- Learn through feedback and consequence instead of answering a quiz.

Evidence of learning A player demonstrates understanding when they track a species, navigate to its habitat, manage a line fight, choose a sale harbour, and buy an upgrade for a stated reason.

Target audience

The primary audience is Year 7-10 students and casual browser players who enjoy collection, upgrades, and short objectives. The first assignment uses plain language and one forgiving fish. Keyboard, pointer, and touch support reduce input barriers; remapping, high contrast, reduced motion, separate volume controls, and pause-on-focus-loss support different access needs.

Planning

Organisation, inspiration, originality, and requirements

Organisation

The team split work by responsibility: game design and programming, visual implementation, progression and documentation, and audio/marine advice. The repository keeps gameplay rules, rendering, input, markets, saving, and platform services in separate modules. Pull requests and automated checks keep the shared main branch playable.

Inspiration and originality

Fishing games often use equipment upgrades, deeper water, and species collection. FSHING keeps those familiar ideas but makes the market the route planner. The player is not completing a catalogue for its own sake: tracking a quote turns the map into a journey from harbour, to habitat, to the better buyer. The title, boat, fish art, maps, interface, prices, animations, and code were produced for this project.

Technical requirements

- Desktop or mobile browser with Canvas 2D and HTML/CSS overlays.
- Keyboard gameplay plus pointer/touch menu and reeling input.
- Fixed 1/120-second simulation step and seeded gameplay randomness.
- Vite and strict TypeScript; Vitest model tests; Playwright browser tests.
- CrazyGames HTML5 SDK v3 behind a local-safe adapter.
- Versioned, validated persistence with safe migration of older saves.

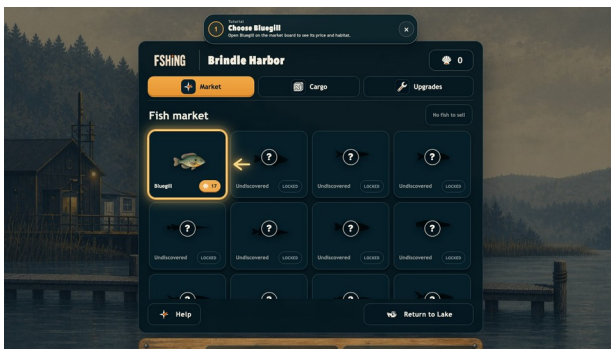
Designing: overview and theme

Game overview

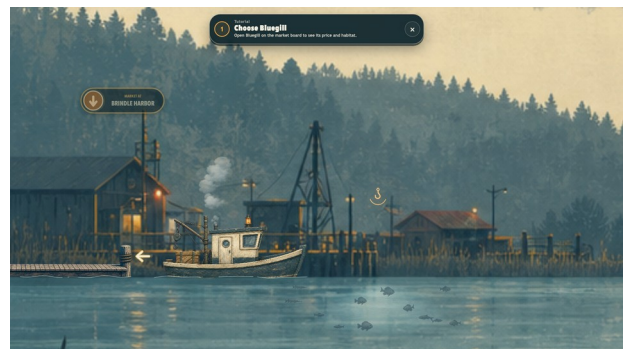
The current build has a lake and an unlockable Beach. Each world has two harbours and three fishing grounds. The lake contains twelve species; the Beach contains eleven. The player starts at Brindle Harbor with Bluegill discovered and a four-step First Assignment.

Theme

A quiet working landscape shaped by water, weathered timber, market boards, and real fish. The mood is practical and slightly mysterious rather than cartoonish. The game uses restrained gouache and screen-print texture, deep teal structures, warm cream text, and amber interaction highlights.



Harbour: information and planning



Lake: movement and atmosphere

Core loop

MARKET → TRACK → SAIL → FISH → REEL → SELL → UPGRADE → MARKET Every system feeds the next decision.

Gameplay mechanics and rules

Market

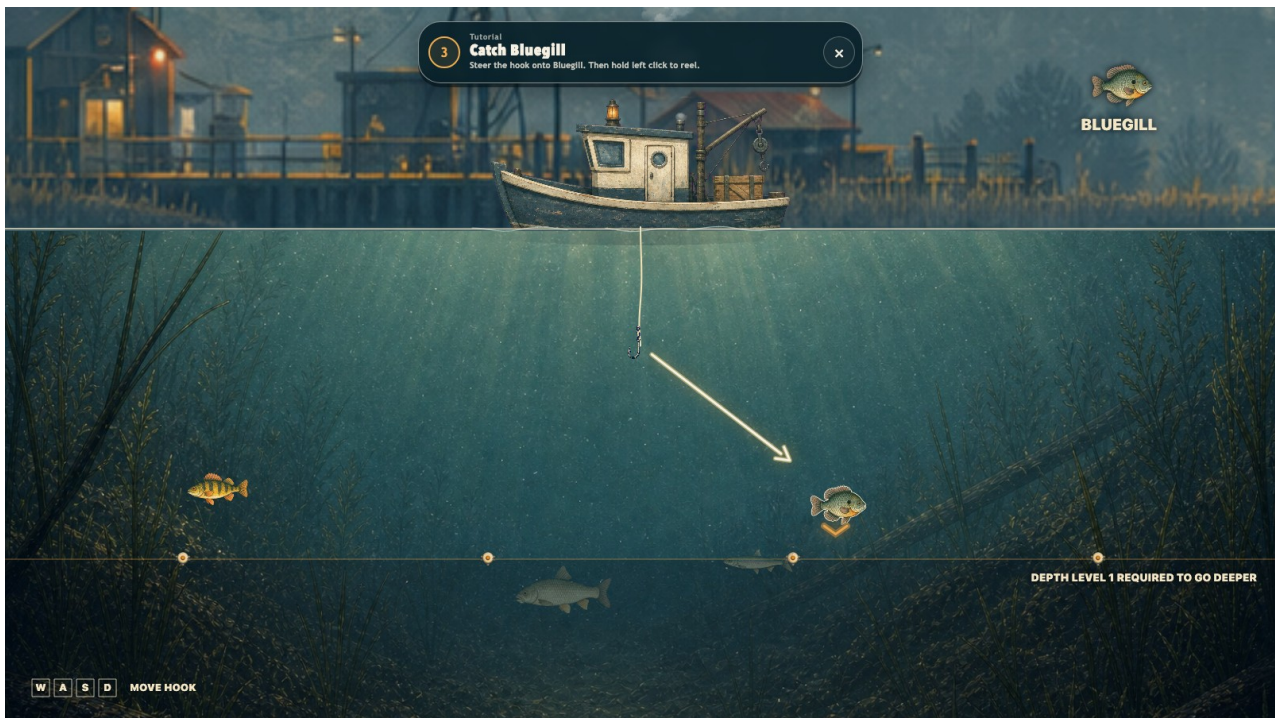
- Each harbour has its own quote for each species; prices step by no more than 6% per day.
- A day lasts 210 seconds of simulation time.
- The detail view shows today's price and a seven-day graph.
- Selling a species sells every cargo catch of that species at the displayed quote.

Sailing and access

- The boat moves horizontally and must slow to dock or drop the line.
- Lake middle/right grounds require line tiers 1/3; Beach middle/right require tiers 3/4.
- Beach costs 300 shells. Oil Rig is visible as Coming soon but has no gameplay.

Fishing

- Steer the hook toward a reachable fish. Depth-locked fish remain visible but dimmed.
- Reel while the fish is calm. Release while it runs so the line can slacken.
- Critical tension snaps the line immediately; the player stays at the ground for another attempt.
- Species use distinct researched movement and fight profiles; starter fish are deliberately forgiving.



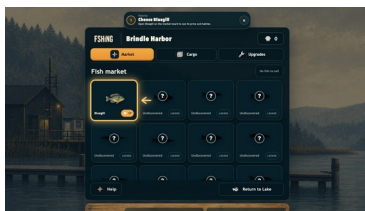
Real gameplay: Bluegill tracked at Sunward Shoal, with hook movement and the line-depth boundary visible.

Storyboard

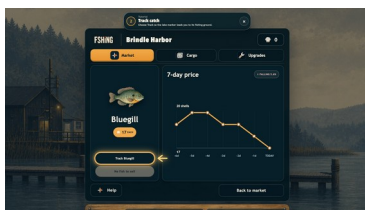
The first-player journey in six frames



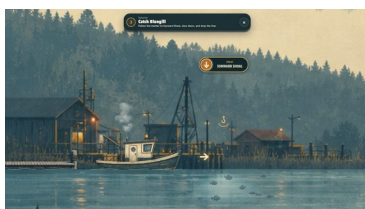
1. Start - Play opens a docked new save.



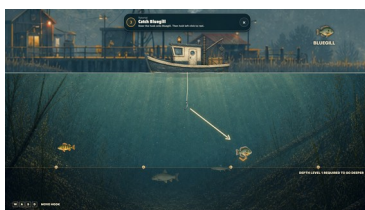
2. Read - First Assignment points to Bluegill.



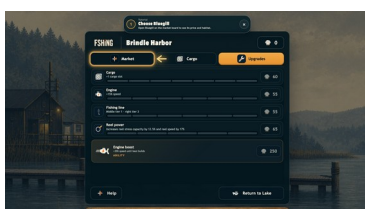
3. Track - Inspect price history and track the catch.



4. Sail - Follow the destination marker to Sunward Shoal.



5. Fish - Steer, reel in calm windows, and release on runs.



6. Grow - Sell, then invest in capacity, speed, depth, or control.

Flow and interface wireframes

Screen flow

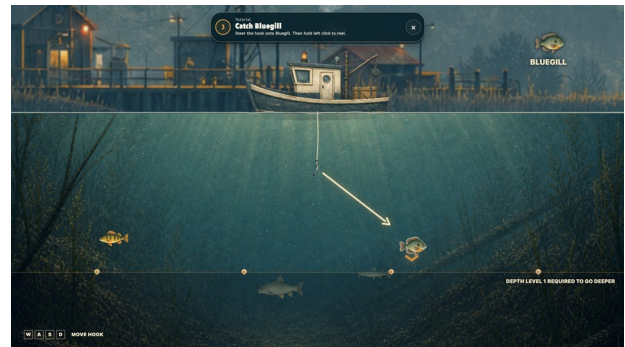
1. Title: Play, Settings, or Credits.
2. Harbour: Market, Cargo, Upgrades, Help, and Departures.
3. Market detail: species art, quote, Track, Sell, and seven-day graph.
4. Lake: destination badge and context action for docking or fishing.
5. Fishing: hook movement, fish silhouettes, line feedback, and tutorial guidance.
6. Pause: Resume, Settings, How to play, or Title screen.

Wireframe decisions

The game keeps permanent on-water chrome small. Money is shown at the harbour, not while sailing. During a fish fight the line itself communicates tension through curve, thickness, and colour instead of adding a meter. Locked fish show a question mark and text, not colour alone.



Information-heavy market detail

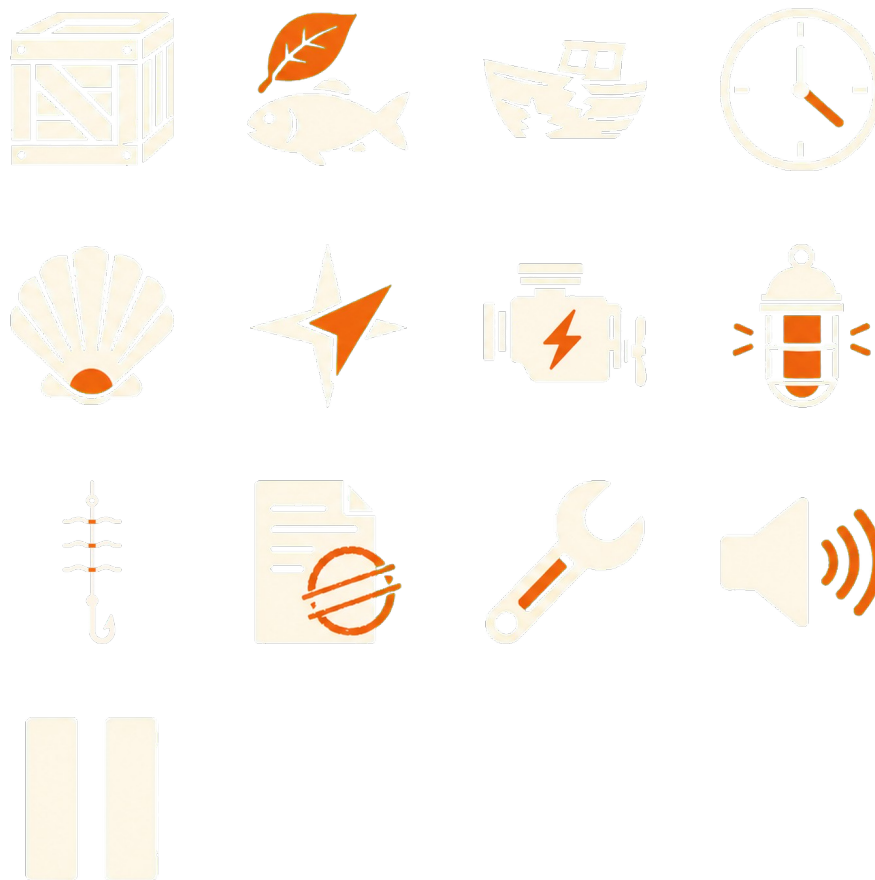


Low-chrome fishing view

Visual and audio design

Visual system

- Correct FSHING wordmark: warm cream capitals, deep teal shadow, small orange accent on the i.
- Deep teal panels separate decisions from the painted world.
- Amber marks the next action; cream carries readable labels.
- Fish use species silhouettes, names, and movement as well as colour.
- Lake and Beach have different panoramas, piers, underwater paintings, and fish sets.



Production FSHING icon atlas: cargo, fish, route, engine, lantern, depth, document, upgrade, sound, and pause symbols.

Audio plan

Quiet title music establishes the working-lake mood. Engine, cast, catch, sale, upgrade, deny, and dock cues confirm actions. Separate saved music and sound-effects volume controls allow independent adjustment. Every important audio cue also has a visible result, label, toast, or motion change.

Asset planning

Runtime asset groups

Group	Examples	Production rule
Brand/UI	Wordmark, logo, panel, button, icon atlas	PNG; transparent or keyed where needed
World	Lake/Beach panoramas, harbours, piers, underwater plates	Full-width paintings; JPG/PNG
Characters	Boat, cargo variant, fish swim sheets, UI portraits	Side-profile sprites and atlases
Effects	Steam, seagulls, surface cues, line-limit float	Small reusable atlases
Audio	Title music plus synthesised interaction cues	MP3 and Web Audio

Production rules

- Runtime assets are explicitly imported so authoring files do not enter the build by accident.
- Each generated asset has a documented prompt, dimensions, role, and processing notes in Docs/Asset-Manifest.md.
- Sprites are tested at gameplay size for silhouette and contrast, not only at source resolution.
- FSHING branding is reused consistently; no replacement logos or unrelated visual marks are introduced.

Testing and fixing

Automated checks

Strict TypeScript checks catch type and contract errors. Vitest covers deterministic movement, fishing, markets, progression, saves, and presentation rules. Playwright covers the playable browser flow, input, menus, accessibility options, SDK fallback, and complete first-assignment behaviour.

Human playtest protocol

Measure	Method	Success target
First sale	Can a new player finish without verbal help?	3/3 testers
Time	Measure launch to first sale.	Under 4 minutes
Market choice	Ask why the player chose a harbour.	Mentions price
Line fight	Observe hold/release behaviour.	Responds to a run
Accessibility	Try keyboard-only and reduced motion.	No blocked action

Fixing loop

REPRODUCE → ISOLATE → FIX → TEST → PLAY A change is not finished until the relevant automated check and the affected player path both pass.

Reflection: feedback already visible in the game

The rows below record feedback already documented by the team and the change that remains visible in the current build. They do not claim feedback from the upcoming incursion.

Feedback	Implemented response
The map should feel larger.	The camera shows about 0.30 world width, making the harbour span several screens.
Use a normal, readable menu.	The title uses one dominant Play action with Settings and Credits beneath it.
Fishing spots should feel alive.	Surface schools, a clear-water lens, and resident fish make each ground visible before casting.
Water should become deeper through upgrades.	Line tiers gate depth and the middle/right grounds.
Add more upgrades and places.	Cargo, engine, line, reel power, boost, and Beach access now form the progression.

What the team learned

The strongest improvements removed explanation rather than adding more interface. Tracking turns a market choice into a route; the tutorial changes wording at the exact moment the player's action changes; the fishing line carries tension feedback without a second meter. The game is clearer when one system explains the next.

Incursion peer-feedback record

Complete during peer review and playtesting

Tester / group: _____ Date: _____

What did the player understand without help?

Where did they hesitate, become confused, or stop?

What was the strongest part of the game?

What single change would improve the next build?

Observed result: ☐ First sale completed ☐ Market compared ☐ Line run handled ☐ Needed help

Feedback-to-change statement

Complete after the incursion

Feedback selected: _____

Why we selected it: _____

Change implemented: _____

Evidence before and after: _____

Verification performed: _____

Do not fill this from memory Attach the peer's exact comment, a screenshot or observation from the test, and the check used to confirm the improvement.

Checklist mapping

Checklist requirement	Where it is covered
Elevator pitch	Page 2; matching presentation deck
Main character description	Page 3
Game concept and learning purpose	Page 4
Target audience justification	Page 4
Planning: organisation, inspiration, originality, technical requirements	Page 5
Designing: overview, theme, gameplay/mechanics	Pages 6-7
Storyboard, flowcharts, or wireframes	Pages 8-9
Visual and audio design	Page 10
Asset planning	Page 11
Testing and fixing	Page 12
Reflection and existing feedback	Page 13
Incursion feedback and change statement	Pages 14-15; complete on/after the day

Bring on the day

- Printed GDD
- Elevator-pitch deck or exported PDF
- Playable current build
- Pens for peer-feedback pages
- A plan to photograph or capture the completed feedback evidence